# Adham Sedik

adhamsedik@cmail.carleton.ca | linkedin.com/in/adham-sedik-971a57184 | github.com/Joyaroin | Ottawa, Canada

## Education

**Carleton University** — Ottawa, Canada
*Bachelor of Computer Science (Honours), Software Engineering Stream* — *Sep 2022 – Dec 2027 (Expected)*

## Technical Skills

**Languages**: Python, C#, TypeScript, JavaScript, Java, C++, C, Bash, HCL
**Backend & Frontend**: .NET 8 (ASP.NET Core), Node.js, Express, GraphQL, REST APIs; React 19, Next.js, Vite, TanStack Query
**Cloud & DevOps**: AWS (EKS, EC2, ECR, Lambda, IAM, VPC, S3, CloudWatch), Docker, Kubernetes, Helm, Argo CD, Kustomize, Terraform, GitHub Actions, Harness, Grafana
**AI & LLMs**: Claude API, Claude Agent SDK, OpenAI API, MCP servers, multi-agent orchestration, structured output schemas
**Databases, Testing & Tools**: PostgreSQL, DynamoDB, Redis; unit, integration and property-based testing, Playwright; Git, Jira

## Experience

**Solace** — Ottawa, Canada
*Software Developer Co-op – Platform Engineering* — *Sep 2026 – Dec 2026*

- Shipped a "disabled" state for offboarded products end to end (UI, API, data model, scripts) in an internal React/Python security dashboard; disabling one stops its scans and new Jira tickets, and a nightly GitHub Actions sweep closes those already filed, **ending 30–60 minutes of hand cleanup per offboarding**.
- Made the EC2 proxies behind CI/CD builds **self-healing**: CloudWatch alarms defined in Terraform reboot an out-of-memory instance while a Lambda posts its status to Slack, turning a failure that struck a few times a month and sat unnoticed for hours into an **unattended recovery**, since proven on an actual outage.
- Provisioned an S3 artifact store in Terraform (bucket, IAM policy, versioning and lifecycle rules) and the build step that publishes a customer-facing product UI to it; now in use, it replaces fetching the latest bundle through CloudFront with **pinned releases that can be rolled back**.
- Wrote a Python reconcile check that fails, with a report, any pull request where a pipeline drifts from its Harness template, and has **blocked one such change before merge**; also added a job testing the CI scripts and refactored workflows into reusable actions.

**Barracuda Networks** — Ottawa, Canada
*Software Engineer Co-op – BarracudaONE Platform* — *Sep 2025 – Aug 2026*

- Delivered C# REST endpoints and .NET service-layer features, plus the React/TypeScript UI on top of them, over **some 60 tickets**, each covered by mock-based unit tests and gated behind a feature flag for safe rollout.
- Built a GitHub Actions workflow that reads the commit currently deployed to production and opens the hotfix branch in the right service repo, removing a manual lookup that risked branching from the wrong state; maintained in one place, it serves **18 service repos**, saving a self-timed **5–10 minutes per ticket**.
- Engineered a test-scenario generator on the Claude API and a Jira MCP server, designing the structured JSON output schema and prompts that turn story subtasks into test cases; **adopted by all 15 developers**, it cut test-note authoring time **roughly 40%**, measured before and after on real stories.
- Authored the Terraform for **production AWS infrastructure** spanning VPC, ALB, IAM, ECR and External Secrets, and set up a Grafana dashboard with threshold-based alerts on SaaS metrics, used day to day by the whole team.
- Raised governance across the service fleet: brought **FOSSA open-source license scanning** into the CI pipeline and a standard pull-request template to **every repo**, and audited three backends against agreed coding conventions.

## Projects

**foundry** | *TypeScript, Claude Agent SDK, Node.js, Git worktrees, GitHub API* — *2026 – Present*

- Created an autonomous pipeline that takes one idea, writes a spec, decomposes it into tickets and implements them without supervision; to date it has **opened and merged 40 pull requests carrying 50 ticket branches** into a personal codebase.
- Designed a scheduler that uses GitHub issue dependencies to compute which tickets are buildable each round, then runs them concurrently under a cap, each builder **isolated in its own git worktree and branch** so parallel agents cannot corrupt one another's work.

**MediaSafe** | *Terraform, Kubernetes (k3s), Argo CD, Helm, Vault, GitHub Actions, Next.js, PostgreSQL* — *2026 – Present*

- Provisioned the VM, firewall, DNS and object storage with Terraform (remote state with locking, credentials injected at run time and never written to disk), and run k3s on it with **every in-cluster change made through Git**: Argo CD app-of-apps with ordered sync waves, Traefik on the Kubernetes Gateway API, cert-manager TLS and default-deny NetworkPolicies.
- Built the delivery pipeline: each merge pushes an image to GHCR and commits its tag to the GitOps repo; Argo CD runs **database migrations as a pre-rollout Job**, then a **zero-downtime rolling update**, and prod is promoted with the exact image staging ran. Secrets live in **Vault** (least-privilege Kubernetes auth) via External Secrets, snapshotted nightly with a tested restore.
- Developed the full-stack app, **live at mediasafe.space**: Next.js 16 and React over Postgres (Neon, Drizzle), Google sign-in, a shared Valkey cache and a nightly TMDB sync, with a custom ESLint rule so failed reads never render as empty.